

TRABAJO INTEGRADOR FINAL | CUIDANDO PELUDITOS

Carrera: Tecnicatura Universitaria en Programación a Distancia (TUPaD) - UTN

Instancia: 2.ª Entrega — Diseño y Módulos (Condición de Regular)

Grupo: 176

Integrantes: Macarena Aylen Marinoni | Marianela Paula Valletto

Docente Evaluador: Londero Oscar

Fecha: Septiembre 2026

1. Introducción y Marco Curricular

El presente documento formaliza el diseño arquitectónico, el esquema relacional de datos y la organización modular del backend para la plataforma **Cuidando Peluditos**, atendiendo de manera directa a la devolución de la 1.ª Entrega.

El diseño se articula sobre las competencias adquiridas en el plan de estudios de la TUPaD:

- **Bases de Datos I y II:** Normalización en Tercera Forma Normal (3FN), restricciones de integridad referencial, transacciones ACID con bloqueo pesimista (`SELECT ... FOR UPDATE`) para prevenir sobreventas por concurrencia y versionado de esquema mediante migraciones controladas (Flask-Migrate).
 - **Metodología de Sistemas I y II:** Arquitectura desacoplada basada en principios SOLID, patrones creacionales y de comportamiento, y formalización rigurosa de la máquina de estados de solicitudes.
 - **Gestión de Desarrollo de Software:** Criterios de aceptación bajo sintaxis BDD (*Given/When/Then*), priorización bajo criterio INVEST y delimitación estricta de alcance (MVP).
 - **Programación III y IV:** Implementación en Python y Flask mediante el patrón *Application Factory* y *Blueprints*, persistencia mapeada con Flask-SQLAlchemy y pruebas automáticas con Pytest.
 - **Legislación (Ley N° 25.326):** Principio de minimización en datos de contacto y domicilios exactos, resguardando la privacidad durante la etapa de búsqueda pública.
-

2. Dominio Central y Reglas de Negocio

2.1. Disponibilidad y Modalidades en P0 (Fase 1)

- **Alojamiento:** Reserva por rango de fechas completas (`fecha_desde` a `fecha_hasta`). Cada cuidador define una `capacidad_maxima` entera que limita cuántas mascotas puede albergar simultáneamente en su domicilio.
- **Visitas a Domicilio:** Reserva puntual por día y franja horaria acordada.
- **Declaración de Condiciones de Convivencia:** El cuidador explicita en su perfil qué especies acepta, tamaños tolerados y compatibilidad con otros animales o infantes, confirmando conscientemente la convivencia al aceptar cada solicitud.

2.2. Solicitudes Multi-Mascota del Mismo Dueño

Una única solicitud puede incorporar N mascotas del mismo dueño. Cada animal individual seleccionado descuenta 1 plaza del cupo disponible del cuidador durante ese período.

2.3. Preservación Histórica e Inmutabilidad (Snapshot)

Los datos críticos de la mascota (nombre, especie, cuidados especiales, medicación y contacto de emergencia) se congelan en la entidad intermedia `SOLICITUD_MASCOTA` al crearse la solicitud. Las modificaciones posteriores que el dueño realice en el perfil de su mascota no alterarán los antecedentes de servicios históricos.

2.4. Protocolo ante Imprevistos e Incidencias

Toda solicitud aceptada debe contener de forma obligatoria un contacto de emergencia secundario, veterinario de cabecera y autorización expresa del dueño para traslados de urgencia con límite de gastos preautorizado. Las anomalías durante el servicio se asientan en la entidad `INCIDENCIA` garantizando trazabilidad y auditoría.

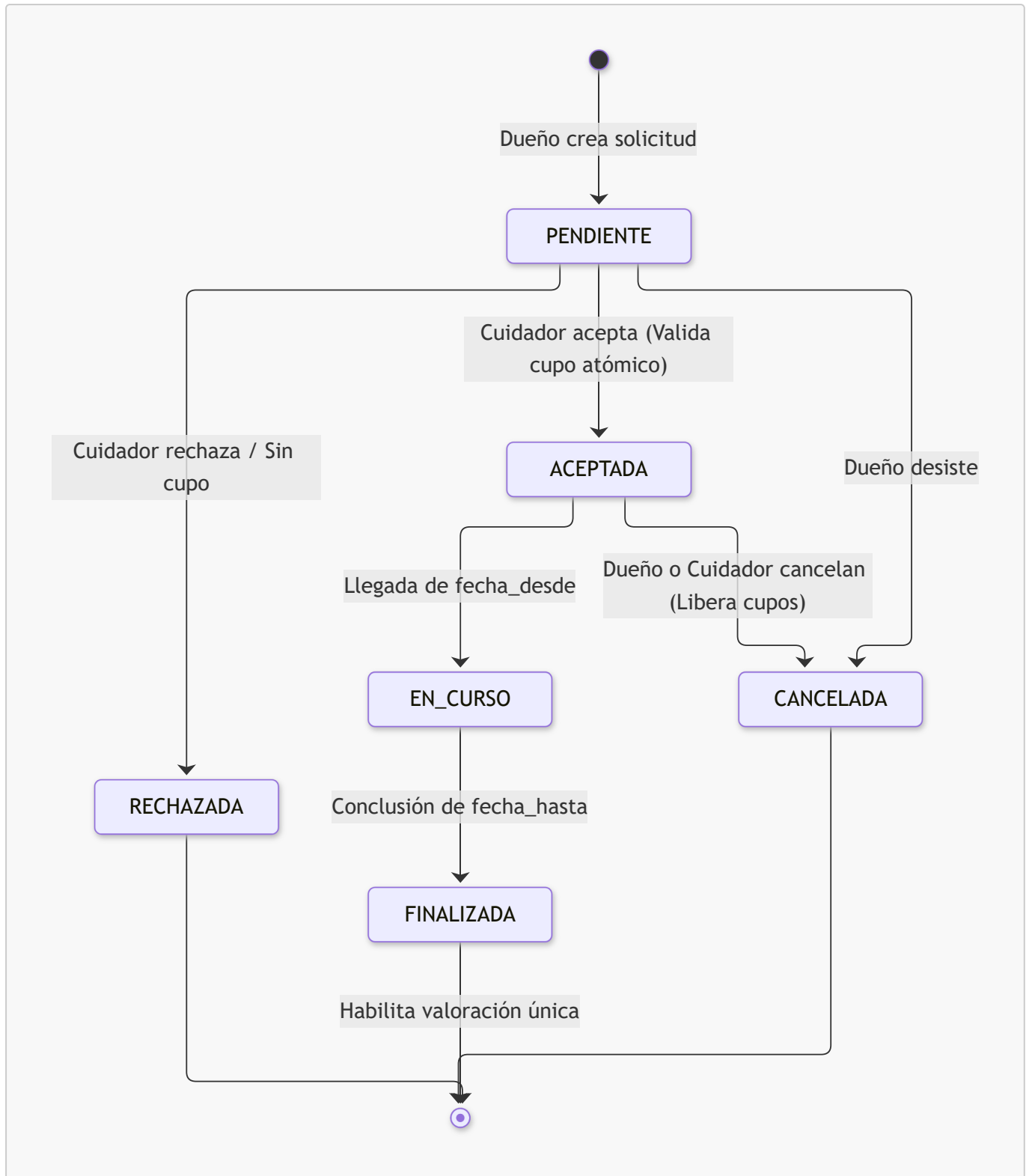
2.5. Concurrencia y Bloqueo de Cupo

La verificación de vacantes se realiza dentro de una transacción en backend con bloqueo de lectura/escritura (`with_for_update()`), asegurando que dos solicitudes aceptadas simultáneamente no superen la capacidad declarada del cuidador.

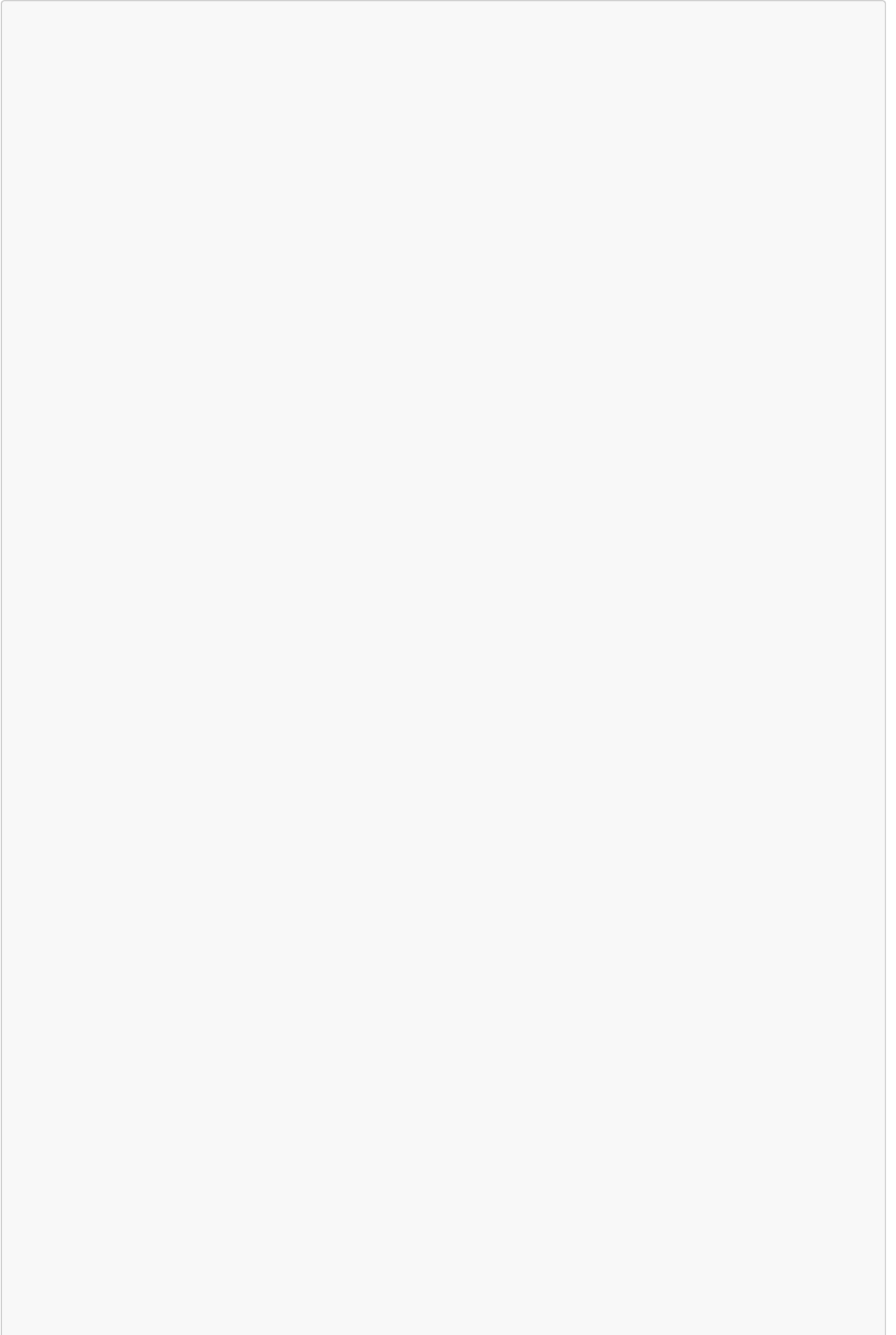
2.6. Valoraciones Verificadas

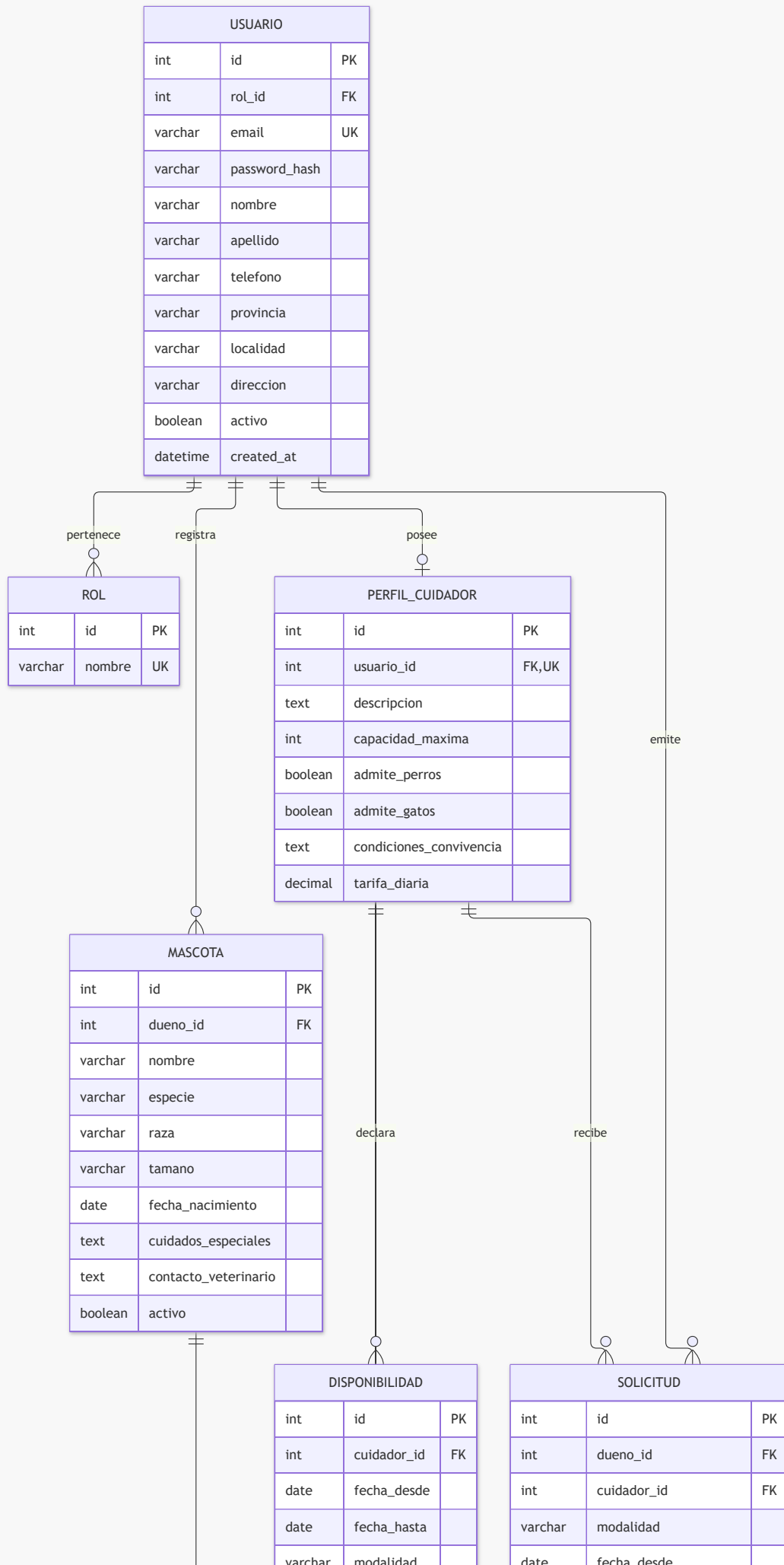
Se restringe la emisión de reseñas y calificaciones exclusivamente al dueño vinculado al servicio, una única vez por solicitud (`UNIQUE`), y únicamente tras alcanzar el estado `FINALIZADA`.

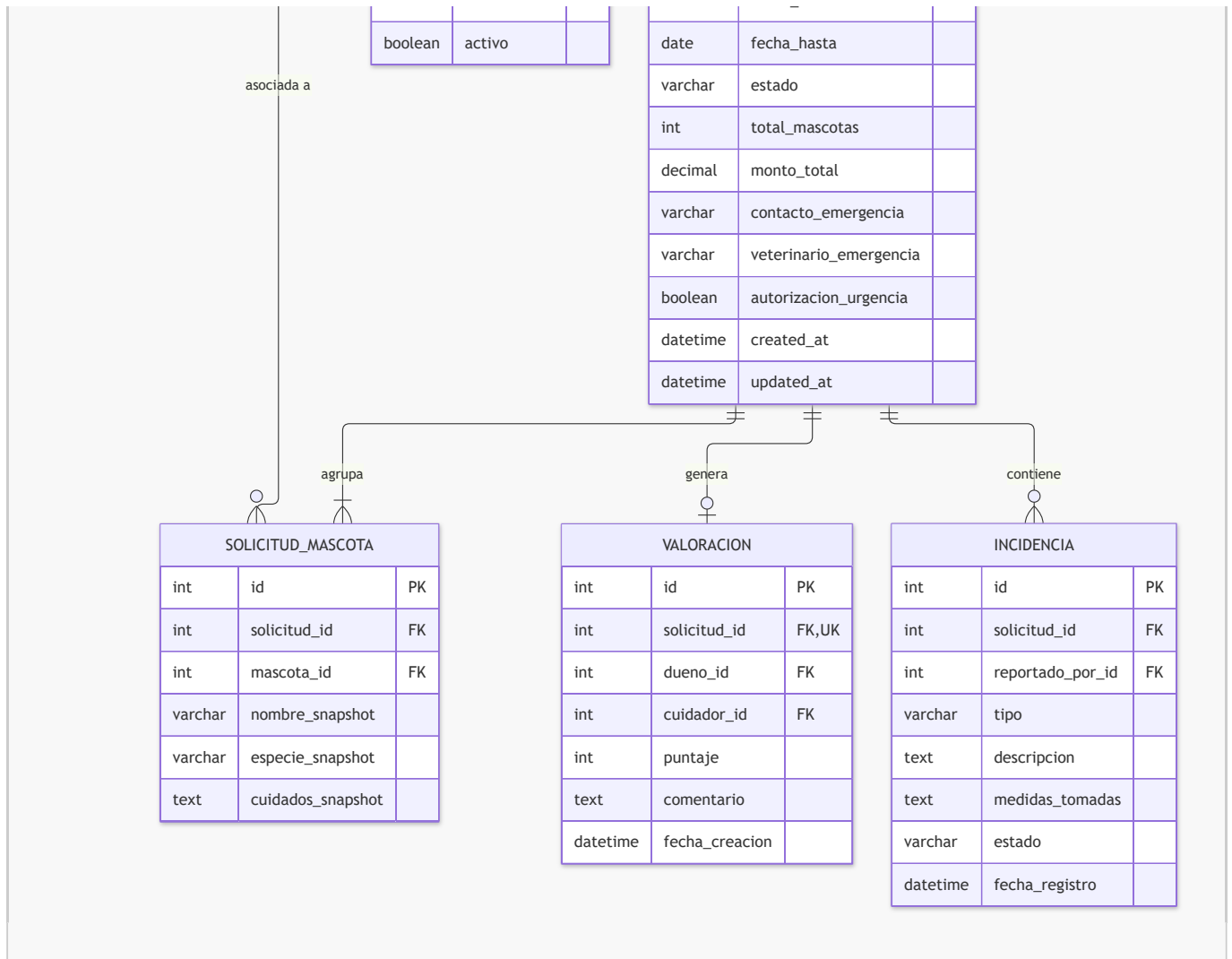
3. Máquina de Estados de la Solicitud



4. MEsquema de Base de Datos Relacional (DER)







5. Estructura Modular del Repositorio (Flask)

Se adopta una arquitectura en capas mediante Application Factory y Blueprints:

app/ | **init.py** # create_app() y registro de Blueprints | **config.py** # Variables de configuración por entorno | **extensions.py** # Instancias de db (SQLAlchemy), migrate y login_manager | **models/** # Capa de entidades ORM | **init.py** | **usuario.py** # Modelos Usuario y Rol | **perfil_cuidador.py** # Modelos PerfilCuidador y Disponibilidad | **mascota.py** # Modelo Mascota | **solicitud.py** # Modelos Solicitud y SolicitudMascota (snapshot) | **valoracion.py** # Modelo Valoracion | **incidencia.py** # Modelo Incidencia | **modules/** # Blueprints de controladores y servicios | **auth/** # Autenticación, sesión y hash de claves | **mascotas/** # CRUD y validaciones de mascotas | **cuidadores/** # Perfil, disponibilidad y búsqueda filtrada | **solicitudes/** # Máquina de estados y transacción atómica de cupos | **valoraciones/** # Registro de reseñas verificadas | **incidencias/** # Gestión y trazabilidad de imprevistos

Modelado del Escenario de Prueba Integral

1. Estado Inicial: El Cuidador C_1 declara disponibilidad del 10/10 al 15/10 con capacidad_maxima = 2.
2. Solicitud Multi-Mascota (D_1): El Dueño D_1 solicita cuidado para 2 perros (M_1 y M_2) del 10/10 al 15/10. Estado: PENDIENTE, total_mascotas = 2.
3. Solicitud Concurrente (D_2): Casi en simultáneo, el Dueño D_2 solicita cuidado para 1 perro (M_3) en las mismas fechas. Estado: PENDIENTE, total_mascotas = 1.
4. Aceptación y Reserva: C_1 acepta la solicitud de D_1 . El backend valida cupo ($0 + 2 \leq 2$), pasa a

ACEPTADA y ocupa 2 vacantes. Bloqueo por Sobrecupo: Al intentar aceptar la solicitud de D_2 , el backend detecta saturación ($2 + 1 > 2$) e impide la transacción, notificando la falta de disponibilidad. Cancelación y Liberación: Con 48 hs de antelación, D_1 cancela el servicio. La solicitud pasa a CANCELADA y se liberan de inmediato las 2 vacantes. Finalización y Valoración: Un servicio previo de D_3 con C_1 alcanza su fecha de fin y pasa a FINALIZADA. El sistema desbloquea para D_3 la posibilidad de emitir una única valoración con puntaje y comentario.